

Pac-Man

Simon Holm, Johannes Rothe og Daniel Nissen

Objectoriented Programming (Java)

Professor: Casper Bach

30th of May 2025

Contents

1	Introduction	4
2	Requirements and Goals	5
3	User Experience	6
4	System Design	7
4.1	UML Diagram	7
4.2	OOP Principles	11
4.2.1	Encapsulation	11
4.2.2	Abstraction	11
4.2.3	Subtype-Polymorphism	12
4.2.4	Interface Polymorphism	12
4.3	SOLID Principles	13
4.3.1	Single Responsibility Principle	13
4.3.2	Open Closed Principle	14
4.3.3	Liskov Substitution Principle	14
4.3.4	Interface Segregation Principle	14
4.3.5	Dependency Inversion Principle	14
4.4	DRY - Don't Repeat Yourself	14
5	Implementation	15
5.1	App.java	15
5.1.1	GameMode.java	15
5.2	GameTimer.java & GameTask.java	15
5.3	Blocks	16
5.3.1	EatableBehavior.java	16
5.4	Map.java	17
5.5	BlockFactory.java	17
5.6	Update.java , Controller.java & Move.java	17
5.7	Eater.java	17
5.8	Revive.java	17
5.9	KillEntity.java	17
5.10	Collision.java	18
5.11	Collidehandler.java	18
5.12	Images	18
5.13	Ghost Pathfinding	19
5.13.1	Grid-based pathfinding	19
5.13.2	Pathfinding algorithms	19
5.13.3	RandomWalk.java	19
5.13.4	BFS.java (Breadth First Search)	20
6	Process	21
7	Quality	22
7.1	Testing each component	22
7.2	Black & White Box Testing	22
7.3	Regression testing	22
7.4	Testing Based On Requirements	22

8	Discussion	22
8.1	EatableBehavior.java	23
8.2	Use of Singleton	23
8.3	Setters	23
8.4	Constants within the code	23
8.5	Design Pattern States	24
8.6	Saving Images In The Entity	24
8.7	Interchangeable map	24
9	References	25
10	Appendix	26

1 Introduction

This project was developed as part of the second semester of the Kunstig Intelligens bachelor programme at Syddansk Universitet, in the course Object-Oriented Programming. The goal of the project is to implement a fully functional version of the classic arcade game PacMan, using core principles of object-oriented design.

In this project, the group has demonstrated through the code a thorough understanding of the ideas behind object-oriented programming, like encapsulation, inheritance, polymorphism, and abstraction. The game has been built in a way that's easy to understand and update later if needed. Each part of the game — like PacMan, the ghosts, the map, the score system, and the game loop — is split into its own class with a specific job, which helps keep the code organized and clean.

Besides using object-oriented programming, this project also provided practice writing clean and readable code, working with event-driven programming, and making different parts of the game interact in real time. One of the more exciting parts was using basic AI concepts to control how the ghosts chase PacMan, making the game feel more dynamic and challenging. Building PacMan has been a fun and hands-on way to turn abstract theory into something real, ensuring the group is ready for more advanced programming and AI work in the semesters ahead.

2 Requirements and Goals

Besides the minimum requirements for this projects, there are a few extra requirements that the group has made.

For the most of this project, the visuals references to the original Pac-Man game, as depicted on Figure 1 from the project description [1].

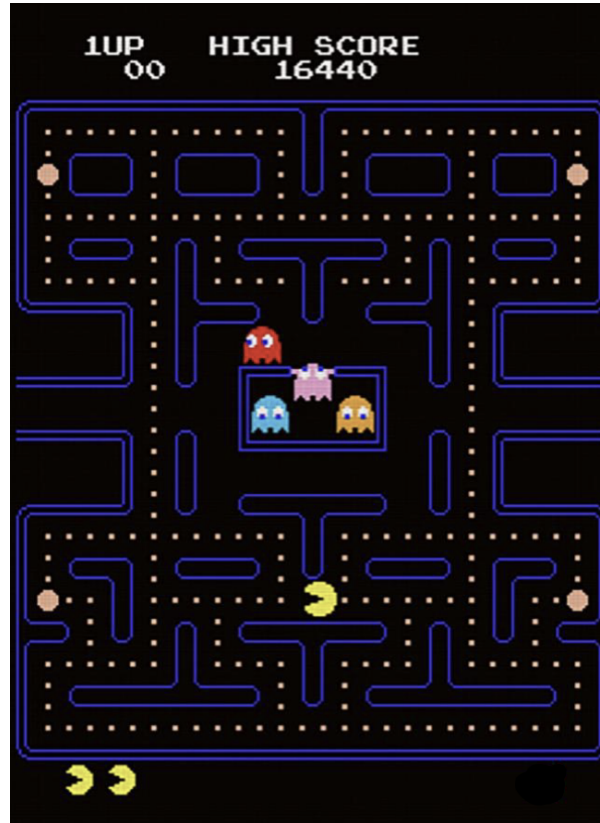


Figure 1: Original Pac-Man

It is a requirement for all ghosts to switch to a power-state in the event that PacMan eats a big pellet, as well as switch to its normal state after some time, no matter if it is eaten or not. The group also added the requirement that an eaten ghost must return to its start position and switch to its normal state on return.

The map itself must be interchangeable. This means that another developer might use a different map, where the layout is different. This should be an option.

3 User Experience

Following the requirements, it is vital to ensure the user experience is considered when creating the game. It should be noted that the game should not just be working theoretically, but should also be enjoyable for the player to play. The following things are therefore taken into consideration.

- Are the controls clear and responsive?
 - The controls (buttons) should feel natural, intuitive, and responsive.
 - The player should be told which buttons to use.
 - The controls must also be responsive enough, so that players find them enjoyable. For this, the control logic must align with what most players expect (intuitive).
- How hard should the game be?
 - The game's difficulty should reflect a basic ability of a player. It should be just difficult enough not to be easy but not hard enough to be unbeatable.
 - The games difficulty should match the levels (so that an average player can just about beat a level, but if practiced, can reach further)
- Visuals should be appealing and easy to understand
 - The game should be visually appealing and easy to interpret. Colors and style choices should support readability while also contributing to the game's overall feel.
 - Important elements such as PacMan, the ghosts, walls, and pellets should be distinguishable from each other.

4 System Design

The structural design of the PacMan game is explained through an overview of the architecture, the main classes, what their responsibilities are, and how object-oriented programming principles have been used. The goal is to show how the system is built to be clear, modular, and easy to maintain, using concepts like encapsulation, inheritance, polymorphism, and abstraction. The design choices focus on making the game work well, while also being easy to expand or change later.

4.1 UML Diagram

To structure the code design, both during the creation process and to get a final overview, UML diagrams were used. One of the greatest benefits of using diagrams during the implementation is the possibility of noticing "anomalies". That is, associations or dependencies that could impede the overall structure. This has been used throughout the project and has been used as guidance on where to improve.

To improve the readability, the full diagram has been split into smaller segments, which will be discussed separately. The full UML diagram can be found in Appendix[1].

The UML Diagram has been split up according to the Model-View-Controller pattern. The MVC pattern defines the logic (model), graphics (view), and input (controller), as distinct parts of the program that should have minimum interdependency. This pattern is followed as much as possible in this project to ensure modularity and flexibility. The first part, Model, can be seen below:

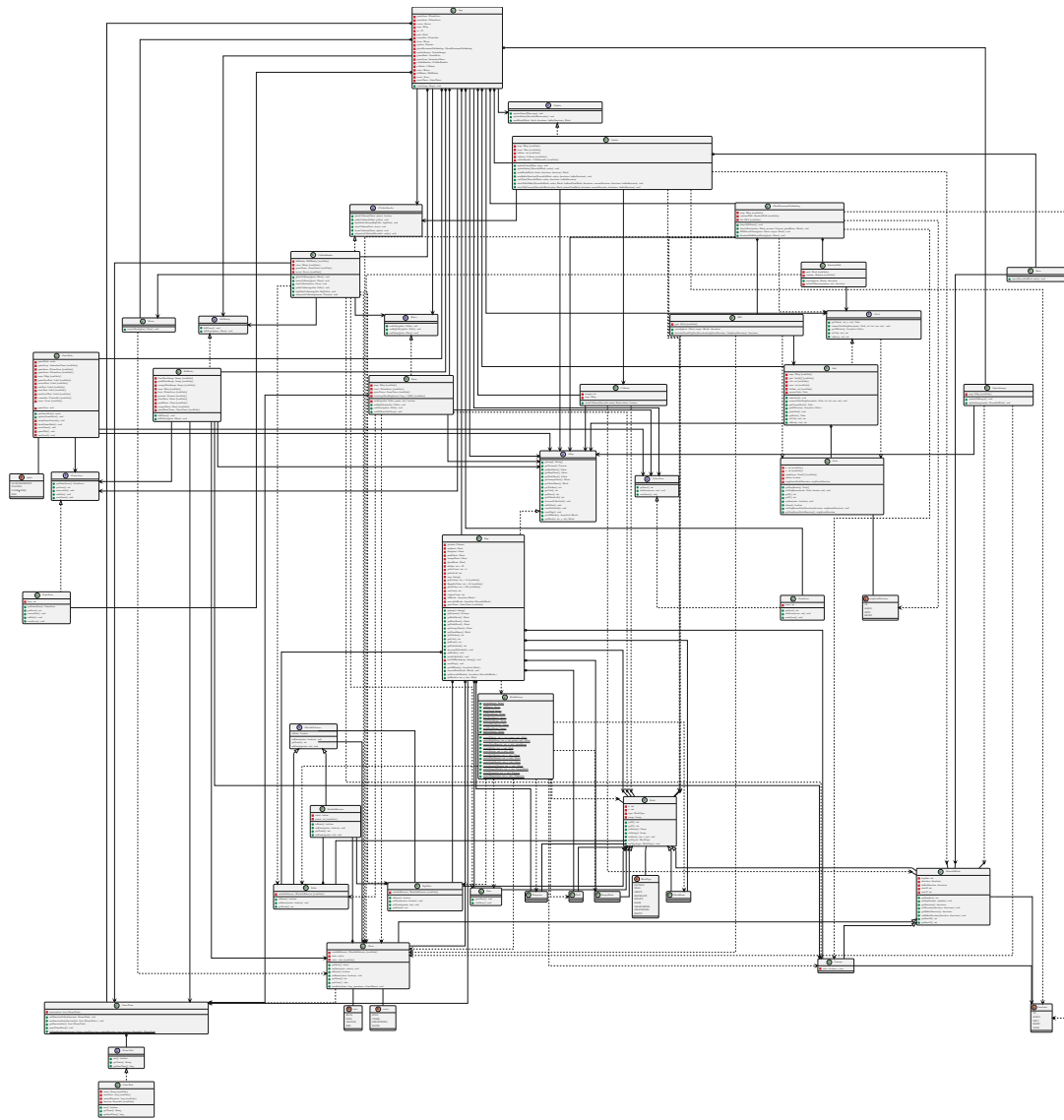


Figure 2: The model part of the UML Diagram

Since the Single Responsibility Principle requires that the full program is split into a host of smaller classes, the full diagram is quite complex. Although there are a lot of important aspects to notice. An important aspect of the model is ensuring its responsibility stays consistent. The model should only be responsible for modifying the internal model of the program based on the input from the user. For example, in the earlier states of the project, **App** was responsible for managing the game UI. This has been changed so that the structure holds the MVC design. Two big composition elements are **App.java** and **Map.java**. Since App's responsibility is initiating all the elements to get the game to run, it's only logical that it's then connected with said elements. The same is true for **Map.java**, since it's a collection of block objects. This can also be seen in the overall layout of the diagram. **App.java** is in the top part, and has a lot of composition arrows from classes that should be initiated. Classes like Update, GameTimer and CollideHandler are initiated and then passed to further classes that need them. At the bottom part of the diagram is the map, and mostly block based classes. Since Map is composed of subclasses of block, most of these classes have a composite arrow to Map, and an inheritance arrow to Block. With Block being a main component of changing the model (for example, getting and setting the block's coordinates), other classes has a lot of dependencies going to it. The Map uses a simple factory, called **BlockFactory.java** to create the blocks stored in Map.java. In this project, it was determined that it wouldn't make sense to implement the full factory method pattern because of the extra complexity, which is not needed. Here, the

`BlockFactory.java` fulfills its role in delegating the creation of blocks' responsibility away from `Map.java`.

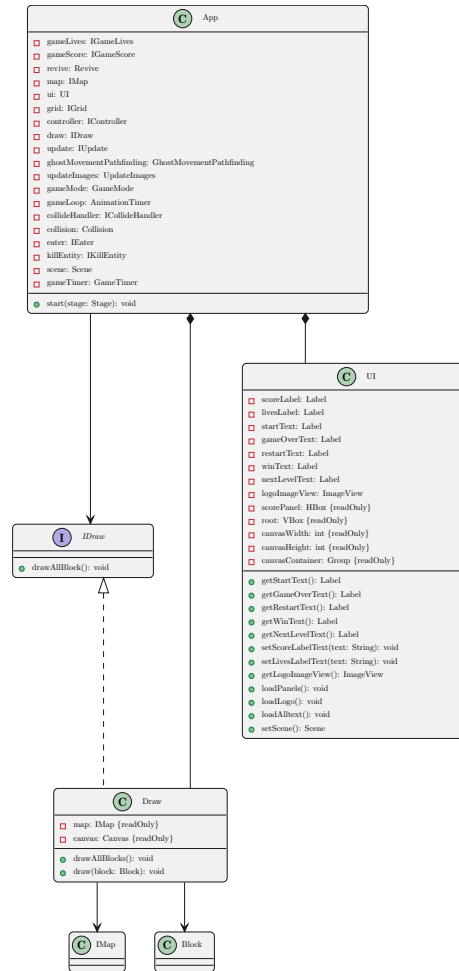


Figure 3: The view part of the UML Diagram

In MVC, the view is responsible for presenting the model to the user. In this case, it's the actual Pacman game. In view, it's important to avoid complex functions that handle data. Its sole responsibility should be to display the model.

To achieve this, some key design choices have been made. First, `Draw.java` only depends on two aspects of the model. It depends on the interface `IMap` to get all blocks, and then `Block.java` to know the position and image the block. By only using the necessary coupling between view and model, unnecessary dependencies and associations can be avoided. This also ensured great flexibility while creating the program and in the future. For example, it doesn't matter what other elements are created in the game, whether it's a powerup or a bigger map. Since all new additions would be subclasses of blocks, they would all always be compatible with view.

Notice that there is no coupling from `UI.java` to the rest of model, except for `App.java`. By using `App.java` to set the text in `UI.java`, its only responsibility becomes displaying the graphical interface, thereby ensuring the MVC principle is held.

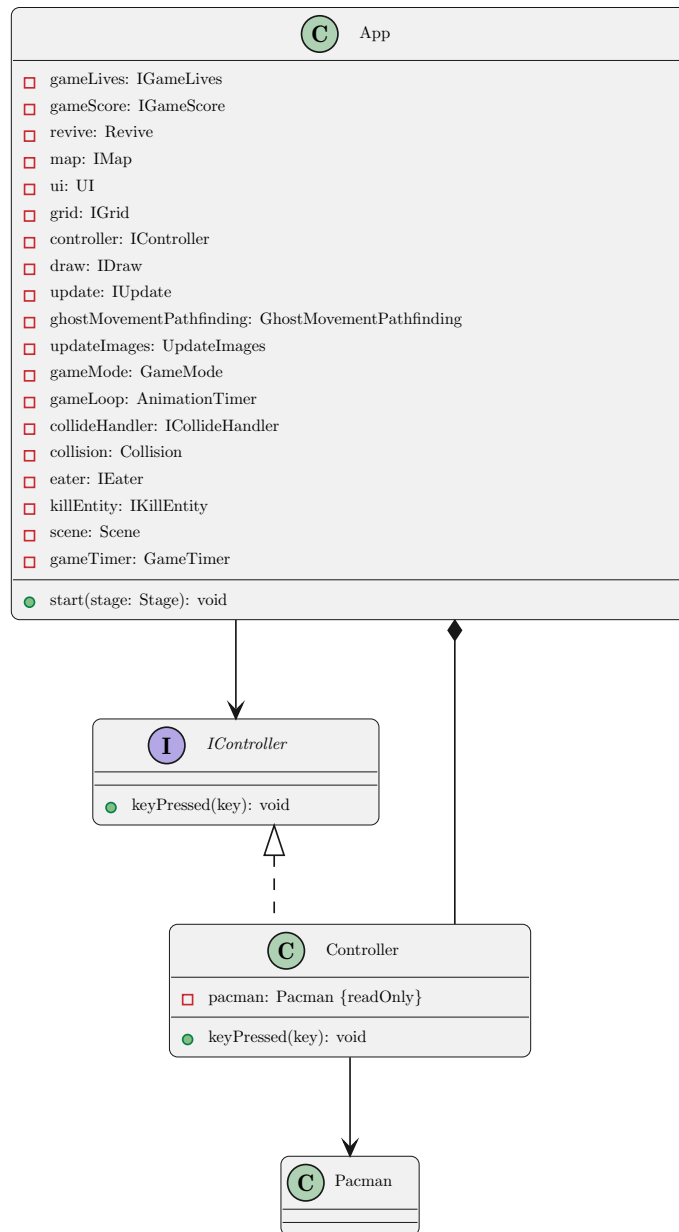


Figure 4: The controller part of the UML Diagram

The Controller is responsible for receiving user input and then redirecting the input accordingly. In some cases, Controller can directly affect the view, but that is not the case for this project. In this project, Controller includes the interface `IController.java` and the concrete implementation `Controller.java`. The input is used in `App.java` to detect when to start the game, and concretely in `Controller.java` to set `bufferDirection` in Pacman. Therefore, the control part of the program doesn't have responsibilities elsewhere, ensuring modularity and flexibility.

4.2 OOP Principles

4.2.1 Encapsulation

Encapsulation is used throughout this project to keep data and logic safely contained within individual classes, so that only relevant parts of the program can access or modify them. This helps reduce complexity, prevent unintended changes, and makes the code easier to manage and understand.

The following example looks at the `map` class:

```
private Pacman pacman;
private Ghost redGhost;
private Ghost blueGhost;
private Ghost pinkGhost;
private Ghost orangeGhost;
private Block ghostHome;
private final int tileSize = 32;
private int pelletCount = 0;
private int pelletsLeft;
```

Notice how the `Map` class uses encapsulation to keep important data and logic hidden inside the class, so that only specific parts of the program can access or modify it. For example, variables like the ones above, and `allBlocks` are marked as `private`, which means they cannot be accessed directly from outside the class. Instead, public methods like `getPacman()`, `getPelletsLeft()`, or `resetMap()` are provided to safely access or update the data. This helps keep the internal state of the game controlled and protected from accidental changes, while still allowing other parts of the program to interact with the map in a clear and limited way. By hiding the details and only exposing what's necessary, the class becomes easier to use and easier to maintain.

4.2.2 Abstraction

Abstraction is used in this project to reduce complexity by exposing only the relevant details of a class or interface, while hiding internal implementation. This allows classes to work together, without understanding the underlying principles the other classes use.

For example, in `Ghost.java`, the ghost's behavior is simplified for external use. The internal attributes for state and color are not accessed directly, instead methods like `getState()` and `getColor()` are used. This ensures that other classes can only retrieve the necessary information without manipulating the ghost's internal logic.

Additionally, interfaces like `IGameScore.java` define a clear contract for how the score should be used. It allows points to be retrieved, added, or reset, but hides how the score is stored internally. This means that the underlying implementation of scoring can be changed without affecting the rest of the program, as long as the interface is used. Similarly, by abstracting "eatable" behavior into an interface, different types of blocks (e.g., pellets or ghosts) can share this common behavior without duplicating code.

4.2.3 Subtype-Polymorphism

Subtypes are used in this project to avoid repeating code by letting classes share common behavior and properties. In the Pac-Man game, a good example is the way characters are structured. Both `Pacman` as well as the different types of `Ghost` are blocks that move around the map, have a position, and use images. Instead of writing the same code in each of these classes, they can inherit from a common superclass like `MoveableBlock.java`, which is an extension of `Block.java`.

```
public class Block {
    // position, image and type

    public class MoveableBlock extends Block {
        // startPosition, stepsize and direction

    public class Pacman extends MoveableBlock {
        // color and state

    public class Ghost extends MoveableBlock {
        // isAlive
```

This means the shared functionality—such as position, movement, or type—can be defined once in the parent classes and then be reused by all subclasses of that parent. Each specific entity can then add its own behavior on top of that. This makes the code cleaner and much easier to update.

4.2.4 Interface Polymorphism

When adding behavior to specific classes, it is good practice to use polymorphism. In the instance of a block being able to "be eaten", all eatable blocks need to inherit behavior from somewhere.

This is achieved using an interface, `IEatableBehavior`, which defines common methods like `isEaten`, `setEaten()`, and `getPoints()` that any eatable block should implement.

Instead of duplicating the eatable logic in every class, each eatable block (such as `BigPellet` and `Ghost`) implements `IEatableBehavior` and delegates the actual logic to a shared helper class, `EatableBehavior`. This class (shown below) contains the logic and information for tracking whether the block has been eaten and how many points it is worth.

```

public class Ghost extends MoveableBlock implements IEatableBehavior {
    private final IEatableBehavior eatableBehavior = new EatableBehavior();

    // Ghost implementation

    @Override
    public boolean isEaten() {
        return eatableBehavior.isEaten();
    }

    @Override
    public void setEaten(boolean eaten) {
        eatableBehavior.setEaten(eaten);
    }

    @Override
    public void setPoints(int points) {
        eatableBehavior.setPoints(points);
    }

    @Override
    public int getPoints() {
        return eatableBehavior.getPoints();
    }
}

```

For example, in the `BigPellet` class, an instance of `EatableBehavior` is used to manage whether the pellet has been eaten. `BigPellet` simply forwards calls to its `eatableBehavior` instance, maintaining clean separation of concerns.

The same pattern is followed in the `Ghost` class. Even though a ghost has more complex logic related to movement and states, its "eatable" behavior (for example, when in `FRIGHTENED` mode) is handled through the shared interface. This ensures consistency and allows the game logic to interact with all eatable entities through the same interface, without needing to know the specific type of object.

4.3 SOLID Principles

4.3.1 Single Responsibility Principle

When designing classes, ensuring that classes doesn't have overlapping responsibilities, or responsibilities they shouldn't have is the core of the Single Responsibility Principle. At the start of the project, `Map.java` was responsible for the game state, points, pacman lives and the creating and managing of the all the blocks in the game. This, however was a violation of the single responsibility principle, so `Map.java` was discussed after the initial implementation. The responsibility of `Map.java` was then clearly defined, it should have all the blocks in the game, and the functionality of initiating/restarting the map but nothing more. As a result of this process, classes like `BlockFactory.java`, `GameLives.java`, `GameScore.java` and `GameMode.java` were used to remove that responsibility from `Map.java`. For example, `BlockFactory.java` ensures that `Map.java` doesn't need to know about the specific complexities of how the blocks are created.

Another example is `Collision.java`, `CollideHandler.java` and `Update.java`. `Update.java` uses `Collision.java` to check for collisions, thereby delegating that responsibility, and in case of a collision, calls the appropriate function in `CollideHandler.java`, so `Update.java` doesn't need to know what should happen, in case of a specific collision. Afterwards, `Update.java` calls `Move.java` that moves that specific entity. This consistent design ensures that all classes have a single, well-defined responsibility throughout the program.

4.3.2 Open Closed Principle

The ideas behind the Open-Closed principle follows principles that had already been chosen to be worked on, such as Abstraction. A good example is the implementation of blocks. The core functionality of blocks is closed for modifications, nothing about how blocks work and interact with the rest of the code should be changed, but it's open for extension, since new block types can be extended from it. This results in a code that's easy to expand. If in the future, a speed boosting block wanted to be implemented, a new subclass could be made, without the need for changing the core functionality of `Block.java`.

4.3.3 Liskov Substitution Principle

`Block.java` also establishes the Liskov Substitution Principle. But making sure that all subclasses have a use for the functionality associated with both their class and superclass attributes. In the earlier stages of the project, the `Block.java` class had the attribute `direction`, which held information on the block's current direction. For non-moving blocks like `Wall.java`, the direction was then just set to `none`. This would imply that the functionality of movement in the program would also apply to `Wall.java`, which it clearly shouldn't. Therefore, this functionality was refactored into a `MoveableBlock.java` subclass, which then has functionality for movement, for which all its subclasses have a use for. This ensures that for every time an instance of a superclass is expected, a subclass can be used.

4.3.4 Interface Segregation Principle

The Interface Segregation Principle states that clients shouldn't rely on methods they do not use. This means in practice, that big interfaces should be avoided. In this project, smaller and more focused interfaces have been preferred for logical reasons. Instead of creating big interfaces and then constantly making sure that every class that uses them uses the full interface, smaller interfaces so every class that needs that specific functionality can decide which interfaces to use was made instead. This might result in more interfaces than strictly necessary, but it has been decided that this is more desirable than breaking the ISP.

4.3.5 Dependency Inversion Principle

Throughout the whole project, the dependency inversion principle ensures that lower level implementations depend on higher level, not the other way around. This means that a class might depend on abstractions (like interfaces), not on concrete implementations. This is also explained further in the Abstraction chapter.

4.4 DRY - Don't Repeat Yourself

A lot of changing and discussion was done within the group to ensure that a lot of code wasn't repeated. As an example a lot of tasks could be collected as one, when all the blocks were made in the beginning of the project, `BigPellets` and `Pellets` were set as each their own type of block, but they both use the same tasks which meant that every method within both types of pellet were made twice, which was very unnecessary. Instead, it was changed so the `BigPellet.java` class would act as an extension of `Pellet` class, so no method had to be repeated but could instead just follow over through the extension of the `Pellet.java` class.

5 Implementation

5.1 App.java

The `App.java` class, is where the game itself is run. The `App.java` class uses JavaFX's graphics and media packages to create the application where the game is shown. Firstly, the start method creates a window for the game. The window size is determined by the map size from the `Map.java` class. The `UI.java` class takes care of all labels, panels, and images (User Interface), and ensures that they are placed within the JavaFX `canvasContainer`. Once all the UI is initially loaded, the `gameLoop` is started. Since the map has already been loaded, the `gameLoop` does the following.

1. Change all ghosts' direction based on their algorithms
2. Update (both move and collision-check) all entities in the game based on their direction
3. Check/change the gamestate (win/gameover/keep-playing)
4. Update all images on the board based on their direction and/or state
5. Draw all blocks based on their position
6. Update the UI (score and lives)

5.1.1 GameMode.java

The `gameMode` refers to what mode the game is in. The game can be in 4 different modes; `NOTSTARTEDYET`, `PLAYING`, `GAME_OVER` or `WIN`. Depending on what mode the game is in, the `updateGameMode()` method is responsible for managing the overall game state. It keeps track of whether the game is in a starting phase, actively running, finished with a win, or over due to failure (gameover). This class doesn't handle gameplay mechanics like losing lives, but it reacts to those changes by updating the game mode accordingly. For example, if the player wins or loses, the class updates the mode and triggers the appropriate reset or end behavior. For the score and lives, separate classes are made for this. Score can be added onto or reset, while lives can only decreased by one or be reset to start value.

5.2 GameTimer.java & GameTask.java

It was decided that the game should have the ability to do things based on time in the game loop, so some implementation was needed to create the possibility to let functions run on a timing-based behavior. To do this, three overall classes were made: an interface `IGameTask.java`, a class implementing that interface called `GameTask.java`, and a class called `GameTimer.java` that uses elements from the `GameTask.java` implementation. The idea behind these three classes is being able to add functions to a list that is constantly checked in the `gameLoop`, ensuring that a function should run after a specified amount of time, so if the amount of time has gone by, the function should be run and removed from the list, it would also remove the function, if there was another one added with the same name. This was useful in cases such as when a big pellet is eaten, because this would make the ghost change back to a `CHASE` state after 5 seconds, but if another big pellet is eaten it should check after the new time, so the player isn't cheated for some extra time they should've gotten. So, the old one would be removed, and a new 5 second timer would be started.

5.3 Blocks

This project is built around the implementation of blocks. Within the game, everything is a type of block, whether it is PacMan himself, one of the four ghosts or a wall, all are blocks. Each block has a type, position, and an image, which can be both seen and altered by getters and setters. This makes it easy to reuse code, and make consistent changes across all blocks if needed. In this regard, the `Block.java` is a superclass that has a lot of specific subclasses. Other essential classes then inherit from `Block.java`, but then add their class-specific behavior. These subclasses can then be grouped into 3 subgroups in which they share behavior. The class `MoveableBlock.java` is a subclass that inherits from `Block`, but then adds more functionality. Since multiple blocks move, it does not make sense to make an individual implementation of a movement system for each class. It is a lot easier to create, maintain, and keep consistent with the rest of the program. Both Ghosts and PacMan needs movement functionality, so by abstracting the specifics to a superclass, the other parts of the code only need to interact with that, instead of the specific classes. `MoveableBlock.java` only has the necessary extra attributes to implement movement functionality, which includes;

- `direction`
- `bufferDirection`
- `stepSize`
- `startX`
- `startY`

`MoveableBlock.java` is then extended by `Pacman.java` and `Ghost.java`. These subclasses then have their own further functionality. `Pacman.java` has an attribute that determines if he's alive, which was deliberately chosen not to use the same implementation for both ghosts and PacMan, due to the difference in use case. Ghosts need a lot more states, like frightened, chase and eaten, which would not be used by PacMan. `Ghost.java` also has more functionality than PacMan, to make the interactability between Ghost and other classes more streamlined, which will be discussed further in the report. The primary extra functionality is the addition of the attributes `states` and `color`, which are used in other parts of the program, which shall be discussed.

The state change has built in, that when the state is set to `Ghost.state.EATEN`, the speed becomes double its base value, mimicking the real PacMan game. The method `resetState()` sets the Ghost's state to `STILL` for a specified time duration, and then changes it to `CHASE`. This is especially useful when creating and resetting the ghosts in `Map.java`.

The other subclasses from `Block.java` are those without movement functionality. They include:

- `Pellet.java`
- `BigPellet.java`
- `Door.java`
- `EmptyBlock.java`
- `Wall.java`
- `Teleporter.java`

`Door` has the extra added methods of being set to open or closed, which changes its picture.

5.3.1 EatableBehavior.java

`EatableBehavior.java` is an implementation of the interface `IEatableBehavior.java` that adds the eatable functionality to objects that should be eatable. Then the blocks use composition to create an instance of `EatableBehavior.java`, which they can delegate the method calls from the interface to. This means that instead of all the blocks needing to implement their individual eatable behavior, one class can handle the logic.

5.4 Map.java

The class `Map.java` is responsible for initiating the map and managing its layout. The map itself is created from a string, and then converted to an `ArrayList` of blocks, each with its own coordinates. The core functionality has then been primarily focused on areas where large parts of the map are involved. The map is created using a `BlockFactory.java`, where the program iterates over the map string, and then adds the appropriate created block to an `ArrayList`. Therefore, `Map.java`'s core responsibility becomes storing the map without having to create each element in it.

5.5 BlockFactory.java

`BlockFactory.java` is a Factory that specializes in building Blocks for `Map.java`. Each method in `BlockFactory.java` is then a creation of an object used by `Map.java`. Since `BlockFactory.java` is a simple factory and the use cases are fixed, creating an interface would add unnecessary complexity without comparable benefits.

5.6 Update.java , Controller.java & Move.java

Another important thing when it comes to games is movement. The player must be able to do something, and not just have to watch the game as if it were a movie playing. For this, an Interface and Class were created, this was `IController.java` and `Controller.java`, which would read the input from the user and apply data to the program based on what was gotten as input. Then a class `Move.java` was made to get to get the data from the controller changes and allows a move in the specified direction upon calling the `move()` function. This was then used in the Interface and Class couple – `IUpdate.java` and `Update.java` which handles all user movement and collisions for the code. It makes sure that PacMan cannot run into walls and calls functions upon collisions with other blocks, and does so for the Ghosts as well. Update is the checker for all movement-related elements, telling the code what to do. If a certain movement is allowed, then it moves the entity it is currently checking for.

5.7 Eater.java

Then it was also necessary to add functionality for handling the specific collision interactions, so they would be callable within the `Update.java` class and follow specific wanted tasks. An Interface and Class were made for the `Eater.java` allowing it to have specific functionalities, that handle what happens when either a pellet, big pellet or ghost is eaten, changing the score and removing the eaten block. For big pellets a functionality to change the state of ghosts were made, making them become frightened and have animations happening after some time showing when the ghost will return to its chase state, where it hunts the player.

5.8 Revive.java

A simple small class `Revive.java` was also made to allow for the ghost to try to be revived if it gets called. This would happen, whenever the `GhostHome.java` is visited by ghost. Then it would check if the ghost had to be revived, meaning that it was in a state of being eaten, and if so, it would be changed into chase once again, putting itself back in the game.

5.9 KillEntity.java

Another set of Interface and Class were made for `KillEntity.java` that had the task to kill either a player or a ghost based on outside behavior and should be able to call a function that would make the appropriate thing happen. As an example, a `killPlayer()` function would be able to remove a life, resetting the game and making sure that every appropriate action would happen when the player gets killed by anything that would call the function.

5.10 Collision.java

The class `Collision.java` was also made. This class had to ensure that a safety margin for collisions could be done, to make collisions visually happen at a point, where it looks normal for the entities to collide and not necessarily need to be straight above each other to let the collision happen.

5.11 Collidehandler.java

Then the class using the methods from these mentioned classes, `CollideHandler.java` was created, and had functions made for each of the collisions that were expected to be able to happen, collisions with ghosts in their different states, collisions with different pellets and for ghosts specifically, collisions with the `GhostHome.java` and `Door.java`. These were all made to implement the different functionalities and allow for a focus-point that update would be able to point to and use for its different methods, when it came to every sort of possible collision it could have registered. And almost every time of collision could be taken straight from this create `CollideHandler.java` Class and be used directly. Ensuring a locked-down functionality that would need to be changed directly for any difference to be made.

5.12 Images

The `UpdateImages.java` class represent all images that needs updating based on either an update from the player (direction change) or an animation. For a pretty and aesthetic PacMan design, it is great for the player, if their actions (such as controller input) have an immediate effect on the game's visuals. In this case, this means updating the entities images such that the player can easily know, which direction the entity is heading. The `UpdateImages.java` has one public method: `updateAllImages()`, which calls `updateImage(entity)` which is responsible for changing entities image depending on their direction and an animation clock, that ensures that entities images are animated. This class only switches an entity's image, this means that has no responsibility in how these images a drawn

The `Draw.java` class on the other hand, has one responsibility only. "Take an entity's image and draw it on the correct positions".

5.13 Ghost Pathfinding

5.13.1 Grid-based pathfinding

The Ghost path-finding relies on creating a node for every walkable tile in the map. This means that any tile that is not a wall, such as open paths, intersections, and corners, is turned into a node. Each of these nodes is then connected to adjacent walkable nodes (up, down, left, right), forming a complete graph that mirrors the layout of the game map. This graph structure allows the ghosts to navigate through the level using path-finding algorithms by treating the map as a network of connected points based on where Pacman and the ghosts are allowed to move.

5.13.2 Pathfinding algorithms

The ghosts' movement is directed by two different walk algorithms, `RandomWalk.java` and `BFS.java`. The algorithms use a `Grid.java` class which is created along with the map. The grid ensures that the algorithms only see nodes, whom all have ≤ 4 nodes as their neighbors. The grid is made in such way that all Nodes have a list of neighbors (e.g. `[null, node2, node3, null]`). This makes it easier for the ghosts to navigate, since for each node they can at all times check its neighbor nodes.

The walk-algorithm both returns a direction. Because of this, the algorithms can work nicely with the ghosts who only need a direction to move.

5.13.3 `RandomWalk.java`

The `RandomWalk` algorithm is a very simple algorithm, since it is not suppose to find PacMan specifically. Unlike `BFS.java` it has no target. The `RandomWalk.java` algorithm firstly uses `random.nextInt(validIndices.size())` to choose a random neighbor node which is not `null`, whenever its direction is `NONE` (standing still). This ensures that whenever the ghost hits a wall, it immediately changes direction according to a valid node.

If the ghost is moving (direction in not `NULL`) a random `double` is used to pick a random direction for the ghosts 50% of the time. This is so that the ghosts will do some turns while moving.

This all ensures that the movement of a ghost using `RandomWalk.java` will seem random.

5.13.4 BFS.java (Breadth First Search)

Much like the `RandomWalk.java` the `BFS.java` algorithm returns a direction and uses the same grid. This time it also keeps track of whether a node has been seen or not. The goal of this BFS is to search for the most optimal route to PacMan. By always keeping track of which direction all possible paths originate from, once the target is found, it can return the desired direction.

The PacMan-BFS algorithm in this case works on a grid (similar to a graph), because of this, it is possible to use the `GRAPHTRAVERSAL()` method. This method is a common "graph-walk", and has a variation, breadth first search, generally good for searching optimal routes in a graph (Figure 5).

```

BFS( $G, s$ )
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 

```

Figure 5: BREADTHFIRSTSEACH as a GRAPHTRAVERSAL

The pseudo-code above depicts the BFS-algorithm as a graph-traversal, it starts of by coloring all vertices white, ensuring that all vertices are able to be seen if reachable. In the PacMan-BFS, this corresponds, to setting all nodes in the grid to not **seen** and their **neighborDirection** to **NONE**. The Graph-BFS then proceeds to color the start node gray, much like the PacMan-BFS which sets the first node to **seen** and sets all its neighbors to their corresponding direction.

Now the actual search begins. In the PacMan-BFS this means that it builds a queue of possible nodes that leads to the path to the target. It then selects the first node from the queue as the **currentNode**. It then checks all the current vertex' valid neighbor vertices. If a neighbor is not **seen**, check it and set it to **seen**. This follows the same **while** loop as in the Graph-BFS, where it keeps checking **unseen** nodes until either target is found or there is no **unseen** nodes left. In that case there is no direct path to the target.

6 Process

The overall workflow of the project has followed a structured and consistent effort from the whole team to ensure both a well-designed and implemented project, but also a balanced workflow within the team. To ensure maximum creativity and clean design, every part of the program was brainstormed, discussed and test-drawn on whiteboards to visualize the intended functionality of each part. By not dividing every part into separate workflows for each team member, the overall vision of the final product managed to stay consistent.

GitLab has been an essential part of the workflow, enabling the ease of making all parts of the code accessible to the whole team at any time. With informative commits, merging and being able to work on a branch of the project has increased productivity tremendously whenever it wasn't possible for the group to meet in person.

AI has had a minimal, but supportive role in this project. Used correctly, it could help spot flaws, or sup-optimal use of SOLID, DRY and OOP principles in the code. Leading to a more consistent final product.

7 Quality

7.1 Testing each component

In the following segment, how testing has been used in the most critical parts of the program to ensure correctness and stability will be discussed. Key functionalities, such as movement logic, collision detection, scoring, game state transitions, and search strategies, have been tested. This helps catch potential bugs early, verify that changes don't break existing behavior, and ensures that the core features work reliably as the project evolves. It is also vital that every (major) change to the structure is tested before push, and more importantly, before anything is built on top of it.

The groups practical knowledge of automated unit testing remains quite limited. Instead, the use of manual debugging using extensive print statements throughout the development process has been relied on. These print statements helped when tracing changes, monitoring program behavior, and quickly identifying errors as they occurred. While not as systematic or proper as actual unit testing, this approach still made tracking down bugs easier, as well as understanding the effects of the modifications during development.

7.2 Black & White Box Testing

When black & white box testing, it is vital to consider the statements from [section 3: User Experience] to ensure that feedback from either a black- or white-box test, can make the game better. White-box testing refers to the developers (authors of this paper) testing the game throughout the project. While this is, without a doubt, the easiest test to perform, it is very biased. Since a developer likely knows every in and out of the program, doing black-box testing (where the tester is not a developer) allows errors or lacks in the program can be more easily noticed. Throughout the project, friends and/or relatives of the authors have been subject to helping white-box test the program. White-box testing was specifically useful when deciding on difficulty changes.

7.3 Regression testing

For this project a lot of regression testing was done, every time a change was made to ensure that the program would still work, this was done partly by expecting and using the error messages that would be gotten in the terminal upon running, and also by inserting print-statements to look for potential errors or general code mishaps that would happen after changing code. There was done this kind of testing several times and major mistakes often relating to null errors were caught as changes were made or other if statements and switches were made to handle information, this happened quite a few times during changes that the changes would allow for other parts of the program to run in unexpected ways.

7.4 Testing Based On Requirements

While the game was being developed, it was deemed necessary to test based on the different requirements given and make sure that the expected functionality was working. This was done especially at the end of the project to ensure that every single part of the specification was followed and had been involved as required. When doing requirement-based testing, looking at every requirement to see if the code would live up to it, by doing this, smaller mistakes such as not having a wait time after PacMan dies were found. This part of the program was then changed to ensure that this was fulfilled, which created problems with the other classes needing a different implementation of the `GameMode.java`, which was done to ensure that a pause state could be handled correctly.

8 Discussion

This section will discuss possible improvements and modifications, if the project were being redone, or more time was given.

8.1 EatableBehavior.java

The `EatableBehavior.java` class acts as a helper to reduce code duplication (DRY) across all eatable blocks, since they share the same behavior. However, in the current setup, this helper is instantiated directly inside each class, rather than being injected from the outside (through the constructor). This makes it difficult to properly unit test the eatable blocks in isolation. Because it's impossible to replace the internal behavior, any unit test on a class such as `Pellet.java` will also end up testing the logic inside `EatableBehavior.java`. This breaks the principles of unit testing, where each class should ideally be tested in isolation, with external dependencies mocked or faked.

8.2 Use of Singleton

Pacman would be a great candidate for a singleton. Since there should only be one instance of PacMan in the program at all times. In this project, PacMan is only initiated once, and then `Map.java` is managing that instance, using a specific `getPacman()` function as an access point.

By using a Singleton, the reliance on `Map.java` could be reduced, leading to a leaner program and less dependency chaining. It would also guarantee that only one instance of PacMan can exist, avoiding complex errors where multiple instances of PacMan are initiated outside `Map.java`'s control. Using a Singleton would reduce the program's flexibility, since the option for creating multiple PacMan's, for example, to use in a multiplayer game, would not be possible. But since the project focuses on single-player PacMan, this potential drawback does not seem relevant.

8.3 Setters

The encapsulation principle ensures that almost all attributes in the program are private. This allows the implementation and good practice use of getters and setters, which can have the functionality of having built-in "value tests". For example, in `MoveableBlock.java`:

```
public void setStepSize(int stepSize) {
    if (stepSize < 1) {
        this.stepSize = 1;
        return;
    }
    this.stepSize = stepSize;
}
```

This setter tests the `stepSize` value, and if less than 1 (for example, 0 or negative values), sets the `stepSize` to the minimum allowed value. This is good practice for setters, and if given more time, would possibly be implemented for every setter in the program.

8.4 Constants within the code

In a few cases, such as the `margin` constant used in `Collision.java`, values are defined without getters or setters. This is not done because it would probably cause even more coupling. Note, even though this does not pose an issue in the current implementation, introducing getters and setters would be beneficial for future development. For example, exposing such values through getters and setters would make it easier to adjust parameters like `margin` dynamically, potentially through a user interface, without modifying the underlying code.

8.5 Design Pattern States

The possibility of using the design pattern States to manage the different states of the game was discussed in the group. The state design pattern allows all the context specifics of a state to be separated into different classes, thus escaping the use of long conditional statements or switch cases. In the following example:

```
public void updateGameState() {
    if (gameLives.getLives() < 0) {
        gameMode = mode.GAME_OVER;
    } else if (map.getPelletsLeft() == 0) {
        gameMode = mode.WIN;
    } else if (!pacman.isAlive()) {
        gameMode = mode.NOTSTARTEDYET;
    }
    checkGameState();
}
```

The class `GameMode.java` constantly checks to change the game mode based on specific conditions. However, since this is a relatively infrequent occurrence, and the conditional statement is still rather short, it was decided to keep it as is. If in the future, more modes were added, implementing the state design pattern might have relevance.

8.6 Saving Images In The Entity

As an extension to any potentially wanted texture changes, it was thought but not done due to time complications, that it would be more correct and easier for modification if Pacman, Ghosts and every other type of block would hold their own images within their class. This would allow for images of every texture to be changed to match the users specific wants and needs and allowing for a more Object-oriented view. Making sure that values for images of specific blocks would not be specified without the blocks own consent.

8.7 Interchangeable map

Although the current map implementation is somewhat interchangeable, future improvements could include the ability to import maps from external text files. This would allow for easier level design, testing, and switching between layouts without requiring changes to the code itself, thereby allowing for more flexibility.

9 References

- [1] Casper Bach (2025), *DM575: Projektbeskrivelse*.
- [2] John Lewis; William Loftus (2018) *Java Software Solution - Foundations Of Program Design (9th Edition)*
- [3] Anthropic (2025), *Claude 3.7 Sonnet Thinking*.
- [4] Google (2025), *Gemini 2.5 Preview*.

10 Appendix

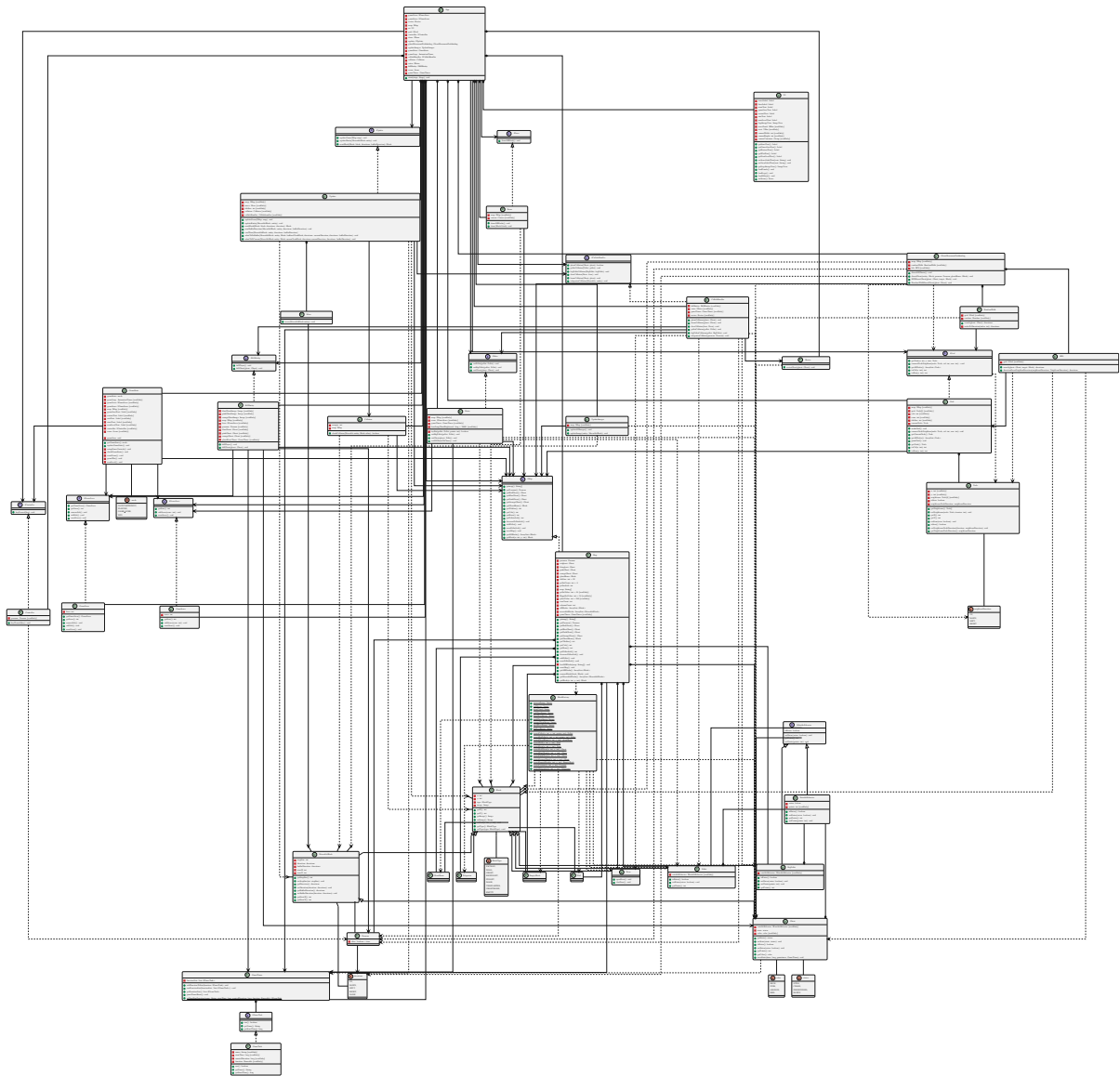


Figure 6: Full UML Diagram of the System